Lecture 11 · Part II · Lecture notes

# Training deep networks

Dataset: CIFAR-10

**How to use these notes**

The primary text for this lecture is Géron, Chapter 11. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

## Contents

---

# 1 A deep stack that will not train

You can build a network and you own its training loop. So build a deep one — twenty hidden layers on CIFAR-10 — and train it exactly as Lecture 10 taught.

It does not train. The loss barely moves. Nothing raises, nothing is `NaN`, the harness is correct and the data is fine, and a shallower version of the same network trains without complaint.

Today we measure what goes wrong, derive why, and repair it — in that order, because the repair is unreadable until you have seen the measurement.

## 1.1 The specification, and what it does not say

3,072 inputs, 20 hidden layers of 100 units, a logistic activation on each, a linear head of 10 outputs. Cross-entropy, Adam at 0.001, batches of 128, 20 epochs.

Read that again and note what it does not mention: *how the weights start out*. That is not something being hidden — it is the ordinary case. Almost no specification mentions it.

| Layer | Shape | Parameters |
|---|---|---|
| First | 3,072 → 100 | 307,300 |
| Each of the other 19 | 100 → 100 | 10,100 |
| Head | 100 → 10 | 1,010 |
| Total | 21 weight matrices | 500,210 |

Comparable to Lecture 9's network, which reached 88% on Fashion-MNIST. Nothing here is under-parameterised.

A model that has learned nothing scores $\ln 10 = 2.3026$. Epoch 1 gives 2.3091 and epoch 20 gives 2.3036: the loss moved by 0.0055 in twenty epochs, and the test accuracy is **10.0%** — exactly what a single `return 3` would have bought.

| Hidden layers | Parameters | Final loss | Test accuracy |
|---|---|---|---|
| 1 | 308,310 | 1.0259 | 41.2% |
| 2 | 318,410 | 1.0503 | 39.3% |
| 5 | 348,710 | 1.6925 | 21.0% |
| 10 | 399,210 | 2.3035 | 10.0% |
| 20 | 500,210 | 2.3036 | 10.0% |

Adding layers made it worse and past a point made it useless — which is not what capacity is supposed to do. And the parameter count barely moves after the first layer, because the 3,072-input layer dominates, so this is not a capacity effect.

The loop is correct, the data is correct, the metric is correct, and the depth is the variable. Which is where Lecture 10's real purchase pays.

## 2 Instrumenting it

Backpropagation computes 500,210 partial derivatives on every batch. Under `MLPClassifier.fit()` every one was discarded; they are all still here in `p.grad`. Two probes, about ten lines each — without them the diagnosis is "it does not train" and there is no next step.

### 2.1 Probe 1, and why it was wrong

Running the modules by hand and recording each logistic's output:

| Layer | Mean | Standard deviation | Saturated |
|---|---|---|---|
| 1 | near 0.50 | 0.1311 | 0.000 |
| 2 | near 0.50 | 0.0707 | 0.000 |
| 5 | near 0.50 | 0.0773 | 0.000 |
| 10 | near 0.50 | 0.0731 | 0.000 |
| 20 | near 0.50 | 0.0710 | 0.000 |

Mean near a half, spread near 0.071, flat for twenty layers, nothing saturated. The forward signal looks fine, and the textbook's first explanation does not apply.

**Failure condition — the probe measured the wrong thing**

`h.std()` spans the *whole tensor*, so it is dominated by the spread of the layer's random biases across units — which does not change with depth. It would read 0.07 in a network that had stopped computing entirely.
What carries information is the spread *down the batch*: how much a unit moves when the input does.

| Hidden layer | 1 | 5 | 10 | 20 |
|---|---|---|---|---|
| `h.std()` — what we plotted | 0.131 | 0.071 | 0.071 | 0.071 |
| `h.std(dim=0).mean()` — the signal | 1.3e−1 | 5.9e−5 | 3.0e−9 | 2.4e−17 |

Sixteen orders of magnitude. The forward signal *does* die, by a factor of 0.149 per layer, and by layer 20 the network is no longer a function of its input.
The saturation column was no better. It asks whether $|h - \frac{1}{2}| > 0.45$ in a band whose standard deviation is 0.071 — a threshold 6.3 standard deviations away, and 3.4 even at layer 1. Nothing could ever have exceeded it. "0.000 at every depth" was arithmetic, not evidence.

## 2.2 Probe 2 — the backward pass

Eight batches rather than one, because a single batch of 128 is a noisy estimate of anything and a course that says so should not then quote one.

| **Layer** | $\|\partial L/\partial W\|$ |
|---|---|
| 1 — nearest the input | 4.567e−17 |
| 5 | 9.296e−15 |
| 10 | 1.845e−10 |
| 15 | 3.042e−06 |
| 20 | 6.608e−02 |
| head | 4.866e−01 |

Layer 20 receives $1.45 \times 10^{15}$ times what layer 1 does.

**The shape of that line is the finding**

It is *straight* on a logarithmic axis, and a straight line on a log axis is a geometric sequence. The attenuation is not one bad layer or a fault at the input: it is the same factor applied at every layer,

$$\frac{\|g_\ell\|}{\|g_{\ell+1}\|} \approx \rho \qquad \Longrightarrow \qquad \frac{\|g_1\|}{\|g_{20}\|} \approx \rho^{19}.$$

One number characterises the whole network, and here it is

$$\rho = 0.1593, \qquad \rho^{19} = 6.91 \times 10^{-16}.$$

Does training rescue it? We logged the per-layer norms every epoch, so this is a lookup rather than a guess: layer 1's gradient was 6.660e−17 at epoch 1 and 4.610e−17 at epoch 20. Twenty epochs of Adam did not move it into a range a learning rate of 0.001 could use. And measuring $\|W_{\text{after}} - W_{\text{before}}\|/\|W_{\text{before}}\|$, layer 1 moved 0.0000, layer 10 moved 0.0001 and layer 20 moved 0.5890. Only the top six layers learned anything, and they had nothing useful beneath them.

**Failure condition — two replies that do not work**

*Adam will compensate.* It rescales, so the direction is what matters — and at $10^{-17}$ the direction is dominated by batch-to-batch noise. Worse, Adam's $\varepsilon$, default $10^{-8}$, is *larger* than the gradients at the bottom of the stack. An optimiser can compensate for a badly scaled gradient; it cannot manufacture information the backward pass did not deliver.
*Raise the learning rate.* The head's gradient is 4.87e−01 and layer 1's is 4.57e−17, so any rate large enough to move layer 1 is $10^{16}$ times too large for the head. A single scalar cannot serve a quantity spanning sixteen orders of magnitude. The repair has to change the *spread*, not the scale.

The chapter names two failures, and they are the same mechanism at different values of one number. Vanishing is $\rho < 1$: the loss does not move, the layers nearest the data suffer, and it is silent. Exploding is $\rho > 1$: `NaN` or a jumping loss, all layers at once, and it is loud. We have the quiet one — and one derivation covers both.

## 3 Derivation: variance propagation through layers

Why 0.1593? Not "why is it small" — why is it *that number*, and what would it have to be instead?

Drop the activation and look at the linear part: $z_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} a_j$. Three assumptions, stated so they can be checked: the weights are drawn independently with mean zero; they are independent of the inputs (true at initialisation, approximately true afterwards); and the inputs have mean zero and are uncorrelated.

All three are idealisations, which is fine — the point of an idealisation is that you can compute with it and then check the answer. Assumption 3 is the weakest at the input layer, since Lecture 9 divided the pixels by 255 so they arrive in $[0, 1]$ with a mean near 0.5.

**The one recursion**

Each term $w_{ij} a_j$ has mean zero and the terms are uncorrelated, so the variances add:

$$\text{Var}(z) = n_{\text{in}} \cdot \text{Var}(w) \cdot \text{Var}(a).$$

Everything in this lecture is a consequence of that line. Note its shape: the input variance is *multiplied* by a factor, which is what makes depth dangerous.

Check it before believing it — one layer, fan-in 100, twenty thousand random inputs of unit variance:

| $\text{Var}(w)$ | **Predicted** $\text{Var}(z)$ | **Measured** |
|---|---|---|
| 0.001 | 0.1000 | 0.1002 |
| 0.050 | 5.0000 | 5.0050 |

### 3.1 Two requirements, and they conflict

The forward pass wants the signal to arrive at layer 20 with the spread it had at layer 1:

$$\text{Var}(z) = \text{Var}(a) \iff n_{\text{in}} \text{Var}(w) = 1 \iff \text{Var}(w) = \frac{1}{n_{\text{in}}}.$$

The backward pass runs the same matrix *transposed*. Using Lecture 10's adjoint notation, $\bar{a} = W^{\mathsf{T}} \bar{z}$, so identical algebra with one index swapped:

$$\text{Var}(\bar{a}) = n_{\text{out}} \cdot \text{Var}(w) \cdot \text{Var}(\bar{z}) \qquad \Longrightarrow \qquad \text{Var}(w) = \frac{1}{n_{\text{out}}}.$$

The index swaps because a row of $W$ and a column of $W$ are different sums: forward, each output unit sums over $n_{\text{in}}$ terms; backward, each input unit collects from $n_{\text{out}}$. That is exactly Lecture 10's fan-in / fan-out bookkeeping — a node's adjoint is the sum over its outgoing edges, and here we are counting how many there are.

| **Layer** | $n_{\text{in}}$ | $n_{\text{out}}$ | **Can both be satisfied?** |
|---|---|---|---|
| First | 3,072 | 100 | no — they differ by a factor of 30 |
| Each hidden | 100 | 100 | yes, exactly |
| Head | 100 | 10 | no |

Every network whose layers change width — which is every network — has to give something up. **Glorot** does not choose, but takes the harmonic mean:

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}},$$

a small error in both directions rather than a large error in one. Harmonic because the quantities being averaged are reciprocals of the fans, and on a square layer it is not a compromise at all — it reduces to $1/n$. On our hidden layers that is $\mathrm{Var}(w) = 0.0100$, a standard deviation of 0.1000, against the default's 0.0577.

## 3.2 Putting the activation back

Between two linear maps sits $\varphi$, and the backward pass multiplies by its derivative pointwise, so

$$\rho = \sqrt{n_{\text{out}} \cdot \mathrm{Var}(w) \cdot \mathbb{E}\left[\varphi'(z)^2\right]}\,.$$

Two factors we control — the shape of the matrix and the variance we initialise with — and one belonging to the activation.

**The logistic's ceiling**

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le \frac{1}{4}$ for *every* $z$, with equality only at $z = 0$. Not "usually small": bounded above by a quarter everywhere.
So the logistic multiplies $\rho$ by at most 0.25 before any weight has been chosen. Even a perfectly initialised logistic stack cannot do better than $(1/4)^{19}$.

ReLU discards exactly half the variance instead: for symmetric zero-mean $z$, $\mathbb{E}[\mathrm{ReLU}'(z)^2] = \frac{1}{2}$ — half the time the derivative is 1, half the time 0. A clean factor of two, and a factor of two can be paid for. **He** pays it by doubling the weight variance, $\mathrm{Var}(w) = 2/n_{\text{in}}$, giving $\rho = \sqrt{n_{\text{out}}/n_{\text{in}}}$ — exactly 1 when the fans are equal. He fixes the *forward* condition, so it uses the fan-in; off square layers, Glorot's compromise is still the honest choice.

| **Scheme** | $\mathrm{Var}(w)$ | $\mathbb{E}[\varphi'^2]$ | **Predicted** $\rho$ |
|---|---|---|---|
| default, logistic | 0.00333 | 0.0625 | 0.1443 |
| Glorot, logistic | 0.01000 | 0.0625 | 0.2500 |
| Glorot, ReLU | 0.01000 | 0.5000 | 0.7071 |
| He, ReLU | 0.02000 | 0.5000 | 1.0000 |

Nothing in that table came from running anything. It is the fan-in, the fan-out, and one expectation — and the first row is the 0.1593 we measured.

**And then it compounds**

$$\frac{\|g_1\|}{\|g_L\|} = \rho^{L-1}$$ $L$ layers, $L-1$ steps between them.

| $\rho$ | over 19 steps | what you see |
|---|---|---|
| 0.144 | $10^{-16}$ | the loss does not move |
| 0.707 | $10^{-3}$ | trains, slowly, badly |
| 1.000 | 1 | every layer learns |
| 1.4 | $10^{3}$ | NaN |

*There is no safe value of $\rho$ except one.* That is what makes deep networks a separate subject rather than a bigger version of shallow ones.

# 4 Seven repairs, measured one at a time

## 4.1 Initialisation, and why it is not enough

| | $\rho$ | End to end | Test accuracy |
|---|---|---|---|
| Default, logistic | 0.1593 | 1.4e+15 | 10.0% |
| Glorot, logistic | 0.25 | 2.2e+11 | 10.0% |

Nearly four orders of magnitude better, and still eleven orders of attenuation — which is hopeless. The measured $\rho$ is now 0.25 — which is *exactly* the logistic's own ceiling, with the weight scale perfectly neutralised. *You cannot initialise your way out of the activation function.*

## 4.2 The activation, and normalisation

Every requirement on $\varphi$ comes from the derivation: $\varphi'$ should reach 1 somewhere and not be bounded below it; $\varphi$ should not saturate on both sides; and the output should be roughly centred so assumption 3 holds at the next layer. ReLU satisfies the first two and fails the third, and is still the right default — though it has its own failure mode, a unit whose pre-activation is negative for every input has gradient exactly zero forever. Leaky ReLU, ELU and SELU all repair that the same way, by giving the negative side a non-zero slope.

| Activation | Initialisation | Test accuracy |
|---|---|---|
| logistic | Glorot | 40.4% |
| tanh | Glorot | 38.5% |
| ReLU | He | 36.8% |
| Leaky ReLU | He | 37.1% |
| ELU | He | 43.2% |
| SELU | Glorot | 41.0% |

Spread 6.4 points — and the logistic is *not last*. All six carry batch normalisation, which is why it survives: normalisation puts the pre-activations back where its derivative is largest, at every

step. Without normalisation, moving off the logistic was worth 31.1 points. With it, the choice of activation is worth 6.4.

Batch normalisation standardises each layer's inputs and then learns what scale they should have had,

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{z} + \beta,$$

where the second step is what stops it being a straitjacket — the network can undo the normalisation if it needs to.

| On the ReLU + He network | Test accuracy | Parameters | Epoch cost |
|---|---|---|---|
| No normalisation | 41.1% | 500,210 | baseline |
| Batch normalisation | 36.8% | 504,210 | 2.16 times the epoch |
| Layer normalisation | 41.6% | 504,210 | cheaper, and still not free |

**Why the repair everyone reaches for first did not pay**

Batch normalisation applied to the *broken* network takes it from 10.0% to 37.9%. Applied to the *already repaired* network it takes 41.1% down to 36.8%, and costs 2.16× the epoch. Normalisation exists to hold $\rho$ near 1 when something else has pushed it away. He initialisation had already put it there, so there was nothing left to correct — and what remained was the cost. On a wider network or a longer run the trade goes the other way. The point is that you can tell which case you are in, because you measured both.

**Failure condition — the bill batch normalisation carries**

Its statistics depend on the other images in the batch, so at training time the prediction for one image depends on which images happened to be batched with it, while at evaluation it uses running averages. Those are two different functions.
That is Lecture 10's `model.eval()` failure again — and now much harder to catch. Dropout wobbles between calls, so running the evaluation twice exposes it; batch normalisation in training mode is deterministic for a fixed batch, so the same trick finds nothing.

## 4.3 Clipping, optimisers and schedules

Gradient clipping rescales the whole gradient vector when its norm exceeds a threshold, preserving the direction and bounding the length. It is a defence against $\rho > 1$, and we have $\rho < 1$: applied alone to the failing network it buys nothing at all. On the repaired network it took 36.8% to 39.6%, but the median step norm with He is 3.19 and the threshold was 1.0 — so it acted as a step-size limiter rather than a safety net. *A useful effect obtained for the wrong reason*; if you want a smaller step, set a smaller learning rate and say so.

Comparing optimisers on the broken network would have measured nothing. On the repaired one: SGD 27.5%, momentum 33.6%, Nesterov 35.2%, RMSProp 36.8%, Adam 36.8%, AdamW 36.1%. The spread between best and worst is far smaller than the effect of the initialisation — order your effort by the measurement, not by the novelty.

Schedules need to know how long the run is, which is why changing the epoch count silently

changes the schedule. Constant 39.6%, cosine 38.8%, 1-cycle 41.5% — on one seed each, so read that as a shape rather than a ranking.

## 5 The ablation

Each row adds one change to the row above. Five seeds a row.

| Configuration | Test accuracy | sd | Change |
|---|---|---|---|
| the failing network, unchanged | 10.0% | ±0.00 | — |
| + Glorot initialisation | 10.0% | ±0.00 | +0.0 (noise) |
| + ReLU and He initialisation | 41.3% | ±0.35 | **+31.3** |
| + batch normalisation | 36.5% | ±0.64 | −4.7 |
| + gradient clipping | 40.8% | ±0.76 | +4.3 |
| + a 1-cycle schedule | 41.3% | ±2.79 | +0.5 (noise) |
| + dropout 0.1 | 34.0% | ±1.33 | −7.3 |

The third column is the deliverable, and the fourth is what makes it readable. Typical spread is 0.84 points; one step is worth +31.3, forty times that. Three more clear it. The 1-cycle schedule does not — +0.5 against ±2.79 — *and the single-seed version of this table called it the winner*.

Applied alone to the failing network: Glorot 10.0%, ReLU with He 41.1%, batch normalisation 37.9%, layer normalisation 10.0%, clipping 10.0%, the schedule 10.0%, dropout 10.0%. Four of the seven leave the network exactly where it was — which is not a disappointment but what tells you what each one is for. Clipping, the schedule and dropout answer problems this network does not have, and Glorot buys eleven orders of magnitude of gradient and is still not enough.

**The answer to "should I add batch normalisation?"**

One change is worth more than the whole ladder. The question is malformed until you know which failure you have — ours was $\rho \neq 1$, and only the repairs that address $\rho$ paid.

### 5.1 The silent-failure catalogue so far

| Failure | Diagnosed in | Measured cost |
|---|---|---|
| Scaler fitted before the split | Lecture 2 | nothing measurable |
| Accuracy under class imbalance | Lecture 3 | a useless model above 90% |
| Missing `optimizer.zero_grad()` | Lecture 10 | 76.7 points |
| Metric averaged per batch | Lecture 10 | 0.382 points |
| A deep stack on default initialisation | today | 31.3 points |
| Glorot's constant used with ReLU | today | 1.5 points |

Every one ran to completion and printed a plausible number. *The last two are the first in which no line of code is wrong.*

## 6 The notebook

**What to change**

Set the initialisation back to the default and re-run the gradient probe. The straight line on a log axis comes back, and its slope is the per-layer factor the derivation predicts — so you will have closed the loop from measurement to theory to measurement yourself.

## 7 Exercises

1. One layer multiplies the standard deviation of the backward signal by $\rho = \sqrt{n_{\text{out}} \operatorname{Var}(w) \mathbb{E}[\varphi'^2]}$. State what $L$ layers do to it, and why only $\rho = 1$ survives depth. [5]

2. Preserving the forward signal asks for $\operatorname{Var}(w) = 1/n_{\text{in}}$ and preserving the gradient asks for $1/n_{\text{out}}$. State when the two agree, and what Glorot does when they do not. [4]

3. He initialisation doubles the weight variance for ReLU. Derive the factor of two in one line. [4]

4. A gradient profile that is a straight line on a log axis tells you something a curved one does not. State what, and why it identifies the cause. [4]

5. Gradient clipping is applied to a network whose gradients are fifteen orders of magnitude too small. Predict the effect, and say what the attempt reveals about the diagnosis. [4]

## Summary

A twenty-layer stack sat at chance while nothing raised. Instrumenting it — which is what owning the loop bought — showed the gradient shrinking by a constant factor per layer, a straight line on a log axis, so one number characterises the whole network: $\rho = 0.1593$, compounding to $1.45 \times 10^{15}$ over nineteen steps.

The first probe was wrong, and the notes keep it. `h.std()` spans the whole tensor and reads 0.07 in a network that has stopped computing; the spread down the batch falls by sixteen orders of magnitude. And a saturation threshold 6.3 standard deviations from the mean returned 0.000 at every depth, which was arithmetic rather than evidence.

One recursion explains it: $\operatorname{Var}(z) = n \operatorname{Var}(w) \operatorname{Var}(a)$, with the fan-in forwards and the fan-out backwards. The two are the same condition only on a square layer, Glorot takes their harmonic mean, and He doubles it because ReLU discards half the variance. The predicted $\rho$ for the four schemes comes from the shapes alone, and the first row is the number we measured.

Seven repairs, each measured alone and then in a ladder over five seeds: the step worth +31.3 points is ReLU with He, against a typical spread of 0.84. Batch normalisation *cost* 4.7 points on a network that no longer needed it while doubling the epoch, and the 1-cycle schedule's +0.5 against ±2.79 was called a winner by the single-seed version of the same table.